AI-Driven Social Media Audience Intelligence Framework

Revised Project Brief v4.8 (Academic Prototype — Hardware Realism & Offline-First Scope)

7th Semester CSE Minor Project | Team of 4 | Sep – Dec 2026

Reference Hardware: NVIDIA GTX 1650 Ti (4GB VRAM) | 16GB RAM | 1TB SSD | Offline-First

> **Document status:** This is the canonical specification. All implementation decisions must trace back to this document. When in doubt, this document wins over verbal agreements or individual assumptions.
>
> **Positioning (locked):** This is an **academic proof-of-concept (PoC) and research testbed** for a CSE minor project. NTRO is the **public problem-statement source only**, not a customer, deployment target, or claim of operational intelligence capability. Do not brand the system as an NTRO product or a production defense SOCMINT platform.
>
> **The Cardinal Product Rule (Offline-First Invariant): "Build the offline demo as the primary product; treat live Telegram collection and cloud LLMs as optional depth."** Campus Wi-Fi drops, Telegram FloodWait rate limits, and missing API keys will kill a live demo. The platform's primary deliverable is a deterministic, zero-internet offline seed data replay (`backfill.py --offline` / SQL seed) and air-gapped local LLM inference via Ollama (`llama3.2:3b`). Live Pyrofork MTProto ingestion is a secondary showcase if connectivity permits.

---

1. Project Identity

Official title: AI-Driven Social Media Audience Intelligence Framework **Problem source (motivation only):** NTRO (National Technical Research Organisation), India — public audience intelligence problem statement. The implemented system is a student research testbed on public Telegram data, not an NTRO system. **Team size:** 4 members **Timeline:** ~15 weeks (first week of September → mid-December) **Internal review checkpoints:** 3 (approximately weeks 4, 9, and 13) **Deliverable:** Fully working system + written report + viva defense (every team member defends every part)

---

2. What This System Does — Plain Language

The system monitors a defined set of public Telegram channels around a selected topic (e.g., Indian space/tech policy), continuously ingests their content, and runs five analytical agents against the collected data to answer questions like:

- "*How is public sentiment shifting around this topic over the past week?*"
- "*Which topics are gaining momentum fastest right now?*"
- "*Are there accounts that appear to be coordinating their posting behavior?*"
- "*What does the inferred audience demographic look like?*"
- "*What is driving the spike in anxiety-coded posts in the last 48 hours?*"

These questions are answered both through a structured dashboard (charts, graphs, tables) and through a natural-language query interface that retrieves **the system's own agent summaries** (not raw posts) and can **refuse** when retrieval is too weak. This is audience-analysis software for a viva, not a production intelligence service.

---

3. Architectural Overview — Locked Decisions

The following architectural decisions are **locked** — they were stated in the submitted project synopsis and must not be changed without team consensus and documentation of the reason.

3.1 Multi-Agent Architecture

The system is built as **five independently invocable agents** (A–E), coordinated by a **lightweight orchestrator**. The word "Agent" is part of each component's actual identity — not a design metaphor.

Component	Agent Name	Role
A	Ingestion Agent	Data collection, storage, timeline management
B	Sentiment Agent	Emotion detection, stance detection, sarcasm flagging
C	Demographic Agent	Aggregate audience profiling (never individual-level)
D	Trend Agent	Topic detection, velocity/acceleration ranking
E	Network Agent	Link analysis, influence mapping, CIB anomaly detection
—	Orchestrator	Routes data between agents, aggregates cross-component insights
—	RAG Layer	Retrieve agent summaries from pgvector; refuse if cosine distance is too high; one structured LLM pass; audit log

3.2 Naming Convention — Non-Negotiable

Use component letters (A–E) consistently in: - Python module/package names (`ingestion_a/` , `sentiment_b/` , `demographic_c/` , `trend_d/` , `network_e/`) - API route prefixes (`/api/sentiment-b/` , `/api/trends-d/`) - Database table/column comments (`-- Component B output` , `-- serves Component A`) - Documentation and report prose

This ensures every reviewer, teammate, and viva examiner can trace any piece of code back to the problem statement.

3.3 Orchestrator Design

- **Type:** Lightweight custom Python coordinator — no LangGraph, CrewAI, or similar heavy frameworks
- **Trigger model: Scheduled batch via Celery Beat** (configurable intervals per component via `.env`)
- **Two distinct trigger types — explicit design decision:**

Row-level trigger (Component B only): `Every 15 minutes: 1. Ingestion Agent (A) pulls new posts → writes to raw_posts (status = 'pending') 2. Orchestrator queries raw_posts WHERE processing_status = 'pending' 3. Fires Agent B against each pending row (per-post sentiment/emotion/stance) 4. Marks processed rows status = 'done'` `processing_status` ('pending' → 'processing' → 'done' → 'error') applies **only to Component B**. A row's status tracks whether per-post inference has run — nothing else.

Window-level trigger (Components C, D, E — separate, less-frequent schedules): `Component C (demographics): every 60 min → queries raw_posts by timestamp range Component D (topics/trends): every 60 min → queries raw_posts by 48h rolling window Component E (network/CIB): every 120 min → queries raw_posts by configurable window` C/D/E do **not** use `processing_status`. They query by `posted_at` timestamp range (actual message creation time, avoiding polling lag). A post being "done" for B has no meaning for D or E — D and E consume ranges of posts, not discrete batches.

Design decision — full recompute vs. incremental: D (BERTopic) and E (Louvain) perform **full recompute** over their rolling window each tick. This is computationally acceptable at semester scale (hundreds to low thousands of posts) and avoids the complexity of incremental BERTopic/graph updates. If post volume grows, the window-level tick intervals are configurable in `.env`. This trade-off is documented in `ARCHITECTURE.md`.

- **Why batch over real-time:** Simpler to build, debug, and demo. Real-time event-driven requires message broker complexity (Kafka/RabbitMQ) that is overkill for this scope. 15-minute latency is acceptable for audience intelligence analysis.
- **"Run Analysis Now" / 15–25s demo invariant on Reference Hardware (locked definition):** The viva button completes within 15–25 seconds on the reference GPU (NVIDIA GTX 1650 Ti) for a capped pending batch (default `DEMO_B_BATCH_SIZE=50` rows, not 1,000), running C/D/E on a capped 48h window (BERTopic max 500 docs), or refreshing from already-written derived tables. Models execute sequentially with explicit cache clearing (`torch.cuda.empty_cache()`). It does **not** mean a full recompute over thousands of posts.
- **Canonical viva path vs live ingest:** Offline seed replay (`backfill.py --offline`) plus local Ollama is the exam dependency. Live Pyrofork is implemented and documented; it is **not** required for a passing demo if campus networks block Telegram.

---

4. Platforms — Scope Decisions

Platform	Status	Reason
Telegram	✅ Primary, must-work	Live ingestion via Pyrofork (actively maintained Pyrogram fork); public channels only
X/Twitter (live)	❌ Removed from live pipeline	Live scraping violates 2023 ToS; research archive requires application process outside project timeline
X/Twitter (offline benchmark dataset)	✅ Offline benchmark only	Pre-labeled coordination dataset used once in Phase 5 to validate Component E heuristics — not a live data source
Reddit	Future work in report only — **do not implement or scaffold**	Stable API, out of minor-project critical path
YouTube	Future work in report only — **do not implement or scaffold**	YouTube Data API v3, out of minor-project critical path
Instagram/Facebook	❌ Out of scope	API restrictions make reliable ingestion impractical; document as future work only

Report note (required): The report must include a paragraph explaining why X/Twitter was removed from the live pipeline (ToS) and why Reddit/YouTube are stretch goals over Instagram/Facebook (API accessibility). The distinction between "X removed from live scope" and "offline benchmark dataset derived from X data" must be stated clearly — these are different decisions with different justifications.

4.1a Offline CIB Benchmark Dataset (Component E only) — Corrected Provenance

The submitted project synopsis claims Component E's coordination detection is "validated against X's labeled ground truth with precision/recall reported." To honour that claim without reintroducing live X access, we use a Kaggle-hosted benchmark dataset, already downloaded locally as `ioa_users.csv`:

Actual dataset identity (verified — do not describe this as Twitter's raw official archive): - Source: Kaggle dataset `pookiewiggington/twitter-information-operations-classification`, described by its author as containing "legitimate and Russian information operations users/tweets" - Construction: combines confirmed Russian IRA-linked accounts originally disclosed via Twitter's official Information Operations archive (transparency.twitter.com) with a separately-assembled set of organic/legitimate accounts, merged into one file with the `info_op` label distinguishing the two classes (1 = coordinated IRA-linked, 0 = organic) - This mirrors a documented academic pattern: Twitter's official archive releases are positive-only (confirmed coordinated accounts only); researchers separately construct matching organic control sets — commonly by extracting hashtags used by IO accounts and pulling concurrent, topically-matched tweets from unrelated accounts over the same date range. The Kaggle file appears to package an existing version of this two-part construction for direct use. - Verified via manual inspection: both `info_op=0` (organic, contemporary general political discourse) and `info_op=1` (IRA-linked, divisive wedge-issue political content, 2015–2018) rows are genuinely present in the same file, at a scale of 11M+ rows total

Required report language (accurate, not overclaimed): "*Component E's coordination-detection heuristics were validated against a Kaggle-hosted dataset combining confirmed Russian IRA-linked accounts (originally disclosed via Twitter's officially released Information Operations archive) with a separately-constructed organic control set, pre-labeled via the `info_op` field. This is not Twitter's raw official archive release, but a third-party-packaged combination of it with a matched control set — a standard construction pattern in coordination-detection literature, since official archive releases are positive-only and never include organic labels natively.*"

Cross-domain and cross-country generalizability justification (required in report): This benchmark uses a Russian-operation dataset to validate a system built for Indian audience intelligence. Published research on content-based coordination-detection features (posting synchrony, near-duplicate content) has tested these feature types across Russian, Chinese, and Venezuelan operations plus a Reddit dataset, finding they generalize reasonably well across country, campaign, and platform — supporting their applicability here despite the geographic mismatch. State this explicitly rather than leaving the Russia/India mismatch unaddressed.

Dataset columns: | Column | Type | CIB signal it supports | |---|---|---| | `userid` | hashed string | Account-level aggregation | | `tweet_time` | datetime | `posting_synchrony_signal` | | `is_retweet` | boolean | Network behavior pattern | | `quote_count`, `like_count`, `retweet_count` | integer | Engagement profile | | `hashtags`, `urls`, `user_mentions` | integer | Behavioral fingerprint | | `BoW` | word list (lemmatized) | `duplicate_content_signal` | | `info_op` | **boolean** | **Ground-truth label** (coordinated vs. organic) |

Why this is still a good choice despite the corrected description: - `info_op` label is **built into the file** — no manual control set construction needed by the team - Both coordinated (`info_op=1`) and organic (`info_op=0`) accounts are confirmed present in one dataset, verified by manual inspection - Already downloaded — no additional access steps required - Large scale (11M+ rows) supports a statistically meaningful precision/recall evaluation

Known gap — `account_creation_date` not present: The `age_volume_signal` heuristic requires account creation date, which this dataset does not include. Decision: benchmark only `posting_synchrony_signal` and `duplicate_content_signal` against the ground truth. State explicitly in the report: "*age_volume_signal was excluded from this benchmark evaluation due to the absence of account creation dates in the dataset.*" Two signals benchmarked honestly is more credible than three signals with one fabricated proxy.

How to use it:

```
1. Aggregate tweet rows → user level:
   userid → { tweet_times, bow_merged, info_op (any-True → coordinated) }
2. Compute posting_synchrony_signal and duplicate_content_signal per user
3. Composite into anomaly_score
4. Classify: anomaly_score > threshold → coordination_signal
5. Compare to info_op → precision, recall, F1
   Also report: precision-recall curve across thresholds 0.0–1.0
```

- **Not a live source:** This dataset is never ingested by Component A, never touches the live pipeline
- **Stored as:** `benchmarks/cib_ioa/ioa_users.csv` — not in Postgres, not in Docker

This resolves the synopsis vs. brief contradiction: X stays out of the live scope (protecting the Viability claim) while Component E's precision/recall claim in the synopsis is backed by a real, correctly-attributed labeled dataset (protecting the Engineering Depth claim) — accurately described rather than overclaimed as an official raw release.

4.1 Target Entity — Demo Configuration

For development and demonstration purposes, the system will monitor a **curated set of 30 public Telegram channels** around a defined topic (e.g., Indian space, defense tech, and strategic affairs). The team must select and document these channels before Phase 1 begins.

Selection criteria: - All channels must be public (no login required, no private group access) - Channels must have sufficient posting volume (>10–20 posts/day) to generate meaningful analysis - Topic should be stable enough to monitor for weeks (recommended: Indian space/science/tech policy, or a comparable domain agreed by the team) - Channel list is stored in `config/channels.yaml` — **seeds the `monitored_channels` database table on application startup (one-directional)**. All runtime code reads channel configuration from Postgres only. The YAML file is the human-editable source; the database is the operational source. Nothing reads directly from the YAML file at runtime.

MTProto Rate-Limiting & Polling Math: With 30 monitored channels on a single authenticated Pyrofork session (`--concurrency=1`), Celery Beat employs a **continuous staggered polling schedule**: $$\text{Polling Cadence} = \frac{900 \text{ seconds (15-minute window)}}{30 \text{ channels}} = \mathbf{30 \text{ seconds per channel}}$$ Every 30 seconds (with 2–5s randomized jitter), the Celery worker queries one channel using `GetHistory(min_id=last_seen_post_id, limit=20)`. This maintains a global request velocity of ~ 2 requests/minute—virtually indistinguishable from organic human reading, completely eliminating Telegram's `FloodWaitError` threshold while guaranteeing that every channel in the 30-channel registry is refreshed every 15 minutes.

Reference `config/channels.yaml` structure:

```
channels:
  - identifier: "@isro_updates"
    display_name: "ISRO Official Updates"
    topic_tags: ["space", "ISRO", "technology"]
    notes: "Primary public broadcast channel for mission status"
  - identifier: "@indianspace_discussion"
    display_name: "Indian Space Discussion Group"
    topic_tags: ["space", "astronomy", "community"]
    notes: "Public discussion community with user replies"
```

Why this matters: An ingestion agent with no defined target collects nothing. The channel list is a project-level decision that must be locked before any code runs. Keeping one runtime source of truth (Postgres) prevents the dashboard and ingestion agent from reading different channel lists.

Offline Demo Seed Script & Viva Resilience: To ensure system demonstration survives unpredictable viva lab conditions (e.g. university firewalls blocking MTProto ports 443/80/1443, or Telegram API FloodWaits during live testing), a standalone backfill utility is included: - Script: `ingestion_a/backfill.py` - Seed data: `scripts/seed_demo_data.sql` (1,000 real historical posts from the selected channels) - Running `python -m ingestion_a.backfill --offline` replays real historical messages into Postgres with normalized current timestamps. This allows the entire pipeline (orchestrator, ML models B–E, and React dashboard) to execute offline without internet access.

Telegram Desktop JSON Export Import (Historical Bulk Data): To enrich the dataset with months or years of historical channel data beyond what live 15-minute polling can collect in a semester, a standalone import script loads Telegram Desktop JSON exports directly into `raw_posts`: - Script: `ingestion_a/import_exported.py` - Usage: `python -m ingestion_a.import_exported --import-json <path/to/result.json>` - Exports are produced via Telegram Desktop → Settings → Advanced → "Export Chat History" → JSON format. - The importer maps exported JSON fields (`id`, `date`, `from`, `text`, `forwarded_from`, `reply_to_message_id`) to the `raw_posts` schema, using the message's original `date` as `posted_at`. - All imported rows are inserted with `processing_status = 'pending'` so Component B automatically picks them up for sentiment/emotion/stance analysis on the next orchestrator tick. - Deduplication is enforced via the `UNIQUE(platform, platform_post_id)` constraint — re-importing the same export is idempotent. - **OSINT boundary preserved:** Only public channels the user has joined can be exported. No private group or DM data is accessible through this path.

---

5. Technology Stack — Definitive

5.1 Data Ingestion Layer (Component A)

Tool	Purpose
`Pyrofork`	Async Telegram MTProto client for public channel ingestion (live 15-min polling). Actively maintained community fork replacing archived Telethon (Feb 2026) and Pyrogram (Dec 2024).
`ingestion_a/import_exported.py`	Telegram Desktop JSON export importer (historical bulk data, `processing_status = 'pending'`)
`Celery` + `Redis`	Task queue for scheduled/background ingestion jobs
`Celery Beat`	Cron-like scheduler for per-component interval triggers
Adapter interface	All ingestion sources implement a common `BaseIngestionAdapter` — new platforms added without touching downstream code

Pyrofork session file — operational security requirement: Pyrofork generates a `.session` file (SQLite) containing the authenticated MTProto user session. This file is functionally equivalent to a logged-in Telegram account. It must be treated as a first-class secret: - Path defined by `SESSION_FILE_PATH` in `.env` (default: outside the repo directory) - Explicitly listed in `.gitignore` — `*.session` must be in the root `.gitignore` before the first `git init` - Never committed, never shared, never stored on a shared dev machine - If compromised: full account revocation (no token rotation possible with MTProto) - **Celery SQLite concurrency deadlock rule:** SQLite does not support concurrent write access across processes. Celery workers handling ingestion tasks must be isolated with

concurrency capped to 1 (`-c 1` or a dedicated single-worker ingestion queue). If two worker processes open the `.session` file concurrently, SQLite throws `sqlite3.OperationalError: database is locked`, corrupting the session and triggering Telegram flood-waits.

Telegram message ingestion & deletion handling: Pyrofork's deleted message handler requires an active, persistent MTProto socket connection. For this minor project, ingestion operates as a forward-moving batch polling pipeline (every 15 minutes), capturing new posts as they arrive. Active deletion-reconciliation sweeps are deferred to future production hardening; during viva demonstration, historical posts collected in Postgres remain static and available for analysis.

5.2 Storage Layer (Component A)

Tool	Purpose
`PostgreSQL 16`	Primary store — timestamp-indexed, append-only
`pgvector` extension	Vector storage for RAG embeddings (same Postgres instance — no separate vector DB)
`Redis`	Celery broker + caching for agent outputs
`Alembic`	Database migrations
`SQLAlchemy`	ORM for all database interaction
Local filesystem	Raw archive files before Postgres parsing (MinIO optional stretch)

Append-only rule: Raw ingested fields are **immutable** after write. Derived/inferred fields live in separate tables. Every derived field traces back to a source `raw_post_id`. This is a non-negotiable data integrity constraint.

Alembic pgvector extension prerequisite: The initial Alembic migration (`0001_initial.py`) must execute `op.execute('CREATE EXTENSION IF NOT EXISTS vector;')` before any table definition referencing `vector(384)`, preventing `UndefinedObject: type "vector" does not exist` migration failures.

5.3 ML/NLP Processing Layer (Components B, C, D, E)

Component B — Sentiment/Emotion/Stance Agent: - `transformers`, `torch` — model inference - **Simplified 3-model pipeline (no language router required):** - **Sentiment (all text):** `cardiffnlp/twitter-xlm-roberta-base-sentiment` — multilingual sentiment classifier (positive/negative/neutral), fine-tuned on 198M tweets across 8 languages. Handles English and Hinglish (Roman-script code-mixed Hindi-English) natively without routing. - **Emotion (English/Hinglish):** `SamLowe/roberta-base-go_emotions` (GoEmotions taxonomy, 28 classes incl. anxiety, excitement, admiration). For Hinglish posts where GoEmotions confidence is low, emotion is derived from sentiment polarity as a coarse proxy (positive → joy, negative → anger, neutral → neutral). Stated explicitly in report. Do not claim 28-class quality on code-mixed Telegram text. - **Stance (all text):** Zero-Shot NLI, language-agnostic. **Runtime default** `valhalla/distilbart-mnli-12-3`. `facebook/bart-large-mnli` is optional/evaluated, not the default worker model. - **No pure Hindi/Devanagari pipeline:** Target channels produce English and Hinglish content. Pure Devanagari support was evaluated (including L3Cube-Pune's HingBERT/HindiBERT base models pre-trained on 52.9M Hinglish sentences) but excluded — these are base language models requiring task-specific fine-tuning, and the multilingual xlm-roberta provides adequate coverage without additional training. - **Evaluated alternatives (documented for report):** L3Cube-Pune's `hindi-tweets-bert` and `hing-mbert-mixed` models were assessed as potential encoders for Hindi/Hinglish text. While these offer native Hinglish pre-training, they are base models without sentiment classification heads — fine-tuning on a labeled dataset would be required. The team opted for pre-fine-tuned classifiers to minimize engineering risk within the project timeline. - **Model Lifecycle & Sequential VRAM Pipeline:** All ML pipelines are initialized once as worker singletons on Celery worker startup (`celeryd_init`

signal) to prevent per-task cold starts. To fit within the 4 GB VRAM ceiling of our reference GPU (GTX 1650 Ti), models execute in sequential pipeline stages: Sentiment inference → `torch.cuda.empty_cache()` → Emotion inference → `torch.cuda.empty_cache()` → Stance inference → `torch.cuda.empty_cache()`. - **Stance detection — Zero-Shot NLI architecture:** - **Runtime default (viva / reference machine):** `valhalla/distilbart-mnli-12-3` (~260 MB, locked). Lightweight, 3x faster, fits comfortably in 4 GB VRAM. - **Evaluated / optional heavier model:** `facebook/bart-large-mnli` (~1.6 GB) — document in the report as an offline benchmark baseline; do not make it the live worker default. - Switch via `STANCE_MODEL` in `.env`. - Target entity: dynamically injected from the primary topic tag in `config/channels.yaml` (e.g. "Indian space policy", "ISRO missions"). - Hypotheses: - Hypothesis 1: `"This text expresses support for {target_topic}."` → `'supportive'` - Hypothesis 2: `"This text expresses opposition to {target_topic}."` → `'against'` - If probabilities for both are below threshold (< 0.6) or neutral entailment dominates → `'neutral'`, or if contradictory → `'unclear'`. Stance is never conflated with emotion or valence. - **Sarcasm detection (formalized heuristics in `sentiment_b/sarcasm.py`):** Rule-based composite scoring: 1. *Valence-emoji incongruity:* Positive valence words paired with derisive/sarcastic emojis (🙄, 😳, 🤡, 😒). 2. *Intensifier contradiction:* Exaggerated superlative expressions ("Masterstroke", "Genius move", "Wah kya baat") co-occurring with negative stance or negative valence. 3. *Punctuation clustering:* Repeated punctuation clusters (`???`, `!!?`) or alternating capitalization. Flagged with `sarcasm_flag = TRUE` and lower confidence annotation; never presented as a confirmed label. - Fine-tuning: documented as future work unless time permits; pre-trained models are the baseline

Component C — Demographic Profiling Agent: - Bio-text parsing + posting-time-pattern analysis - **Data source & ethical defensibility (Telegram boundary constraint):** Public broadcast channel subscriber lists are private and inaccessible via MTProto API. Demographic profiling is strictly operationalized across two observable tiers: 1. *Tier 1 — Broadcast Channel Level:* Macro geography, primary language, and publishing schedule heatmaps derived from public channel metadata, description bio, and broadcast timestamps. 2. *Tier 2 — Audience Engagement Sampling:* Inferred demographic distributions (age brackets, professional interests, language distributions) are sampled exclusively from **active participants in linked discussion groups and post comment threads** (where user `raw_authors` profiles, bios, and reply timestamps are public and accessible). - **Inference Quality Validation:** Because unlabelled Telegram audiences lack public ground truth, heuristic accuracy is validated via a structured qualitative spot-check of **20–30 public author profiles** with explicit self-disclosures (e.g. self-declared occupation, regional/city indicators, or educational affiliation). Inferred language, geography, and interests are compared against manual annotations to report qualitative concordance and failure modes alongside privacy filter metrics. - Output: aggregate-only, enforced by minimum cluster size guard. **Report / ethics figure: `MIN_CLUSTER_SIZE=50`.** Monitoring 30 channels provides sufficient audience comment engagement across linked discussion groups to populate genuine $k \ge 50$ clusters. **`.env` may lower k for the viva seed** (e.g. 15) if needed for smaller demo replay subsets so the grey-out UX remains demonstrable. Never invent users to hit 50. Never silently hide sub-threshold clusters. - UI label: **Audience aggregates (inferred)** — heuristic bio/keyword inference, not census demographics. - Clusters below threshold: returned with `below_threshold: true` flag; UI greys them out (never silently hidden)

Component D — Trend/Topic Detection Agent: - `bertopic`, `umap-learn`, `hdbscan` — topic modeling - `sentence-transformers` (`paraphrase-multilingual-MiniLM-L12-v2`) — multilingual embeddings (50+ languages, 384-dim); replaces `all-MiniLM-L6-v2` which is English-only - Topics ranked by **velocity** (rate of new posts) and **acceleration** (change in velocity) over Component A's timeline - Time window: configurable (default: 48-hour rolling window). **Cap documents per BERTopic run** (default: 500 most recent posts in the window) so "Run Analysis Now" cannot OOM the worker. - **Single embedding model for the whole system:** `paraphrase-multilingual-MiniLM-L12-v2` (Component D + RAG). Do not load a second sentence-transformer. - **Timestamp used: `posted_at` (not `ingested_at`)** — velocity and acceleration measure actual posting behaviour; `ingested_at` measures polling lag and must never be used for temporal analysis - **Low-volume calibration & outlier handling:** HDBSCAN's

default `min_cluster_size=15` collapses sparse posts into noise (`-1`) during demo-scale runs. Component D configures: - `min_topic_size = 3` (or `5`) for stable topic extraction on moderate-volume channels - `nr_topics = "auto"` - `topic_model.reduce_outliers(docs, topics, strategy="c-tf-idf")` to reassign outlier posts to nearest centroids, guaranteeing zero dropped posts during evaluation.

Component E — Network Analysis & CIB Detection Agent: - `networkx` — graph construction and centrality analysis - `python-louvain` — community detection (Louvain algorithm); **note: installed as `pip install python-louvain` but imported as `import community` in Python — not `import python_louvain`** - **Graph projection constraint for Louvain:** While `network_edges_e` stores directed interactions (`forward`, `reply`, `mention`), the Louvain modularity algorithm is mathematically defined only for undirected graphs. Component E must project the directed interaction graph into an undirected graph with summed bidirectional edge weights (`undirected_G = G.to_undirected(as_view=False)`) before computing community partitions, preventing `TypeError: Bad graph type` runtime crashes. - **Entity model (Channels & Authors):** In Telegram public broadcast networks, the primary interacting entities are **Channels** (broadcasting content and forwarding from other channels) and **Authors** (users participating in discussion threads). Network nodes model entities with an `entity_type` (`'channel'` or `'author'`), preventing null-author insertion failures on broadcast posts. - **Timestamp used: `posted_at` (not `ingested_at`)** — all synchrony calculations use actual message timestamps; same reason as Component D - CIB anomaly signals (**three** heuristic sub-signals, composited into `anomaly_score`): - `posting_synchrony_signal` (s_1) — accounts posting within tight time windows relative to one another (uses `posted_at`) - `age_volume_signal` (s_2) — young account with disproportionately high posting volume - `duplicate_content_signal` (s_3) — near-duplicate content via TF-IDF cosine similarity on post text - **Composite anomaly score formulation:** Heuristic sub-signals are composited using a straightforward unweighted arithmetic mean over available signals: $$\text{anomaly_score} = \frac{s_1 + s_2 + s_3}{N_{\text{available}}}$$ If $\text{anomaly_score} \ge 0.70 \rightarrow \text{coordination_signal} = \text{TRUE}$. This formulation is simple, intuitive, defensible to examiners, and avoids inventing uncalibrated statistical weights on unannotated Telegram data. - **Centrality:** `betweenness_centrality` is the primary sort key for the 500-node graph cap — betweenness surfaces accounts that bridge distinct communities, which is the highest-value signal for detecting coordination brokers; degree centrality alone surfaces the loudest accounts, not the most structurally significant - All live outputs labeled `anomaly_score`, `coordination_signal` — never "confirmed coordinated" or "bot" - **Offline validation (Phase 5):** `posting_synchrony_signal` and `duplicate_content_signal` benchmarked against the offline dataset (Section 4.1a); `age_volume_signal` validated qualitatively on Telegram sample data — stated explicitly in report - `pandas`, `scikit-learn` — used in the offline benchmark script only (`benchmarks/cib_ioa/evaluate.py`)

5.3b Hardware Specifications, Memory Footprint & Sequential VRAM Strategy

To ensure reproducibility across the student team and eliminate the risk of worker crashes during the viva defense, the system is architected around the following hardware profile:

Primary Reference Development & Demo Rig: - **GPU:** NVIDIA GeForce GTX 1650 Ti Mobile (4 GB GDDR6 VRAM) - **System Memory:** 16 GB DDR4 RAM - **Storage:** 1 TB NVMe SSD + 1 TB HDD - **Operating Environment:** Windows 11 / WSL2 / Docker Desktop

Total Model Memory Footprint (4 GB VRAM Budget Analysis): | Component & Model | Disk Weight | VRAM / RAM Footprint | Execution Strategy | |---|---|---|---| | `cardiffnlp/twitter-xlm-roberta-base-sentiment` | ~1.1 GB | ~1.1 GB VRAM | GPU Stage 1 (batch inference) | | `SamLowe/roberta-base-go_emotions` | ~500 MB | ~500 MB VRAM | GPU Stage 2 (batch inference) | | `valhalla/distilbart-mnli-12-3` (Stance) | ~260 MB | ~260 MB VRAM | GPU Stage 3 (batch inference) | | `paraphrase-multilingual-MiniLM-L12-v2` (Embeddings) | ~470 MB | ~470 MB VRAM | Shared by BERTopic & RAG | | Ollama (`llama3.2:3b` 4-bit quantized) | ~2.0 GB | ~2.0 GB VRAM/RAM | Standalone service (offloads to RAM if needed) | | System Services (PostgreSQL 16, pgvector, Redis, Celery, FastAPI, React) | N/A | ~2.5–3.5 GB RAM | System RAM | | OS & Desktop Overhead | N/A | ~3.0 GB RAM | System RAM |

Critical VRAM Invariant — Sequential GPU Pipeline: If all transformer models and Ollama were loaded simultaneously into VRAM, total consumption would exceed **4.3 GB**, immediately triggering a CUDA Out-Of-Memory (OOM) crash on a 4 GB GPU. Therefore: 1. **Sequential Batch Processing:** Celery workers process pending batches through one model at a time, explicitly invoking `torch.cuda.empty_cache()` between pipeline transitions. 2. **DistilBART Locked Default:** Utilizing `distilbart-mnli-12-3` (~260 MB) instead of `bart-large-mnli` (~1.6 GB) saves 1.3 GB of memory and accelerates inference by $3\times$. 3. **Ollama Service Isolation:** Ollama runs in a separate background daemon (`ollama serve`); on 4 GB cards, its runtime automatically balances layers between GPU and system RAM without crashing. 4. **Teammate Fallback Minimum Spec:** For teammates lacking dedicated GPUs, all models support CPU-only execution with quantized weights, and the offline seed replay functions without ML acceleration. 5. **Capped Demo Batch Timing:** On the GTX 1650 Ti, processing a capped batch of ≤ 50 seed posts through the entire pipeline takes **15–25 seconds**, satisfying the viva demonstration requirement without false claims of real-time throughput on massive corpora.

5.4 Backend API Layer

Tool	Purpose
`FastAPI`	REST API framework (async-capable)
`Pydantic v2`	Request/response schema validation (doubles as documentation)
`uvicorn`	ASGI server
Polling (locked)	Dashboard refreshes via HTTP polling. **No WebSockets.**

Authentication: HTTP Basic Auth middleware on all endpoints. Credentials stored in `.env`. Not production-grade — sufficient for demo access control and viva defensibility.

5.5 RAG Query Layer (Corrective RAG & Self-Reflection Contract)

Tool	Purpose
`sentence-transformers` (`paraphrase-multilingual-MiniLM-L12-v2`)	Embedding generation for agent summaries — multilingual, 50+ languages, 384-dim
`pgvector`	Vector similarity search (HNSW cosine distance)
LLM provider (abstracted)	Structured generation with self-reflective output contract

LLM Provider — abstraction layer:

```
class BaseLLMProvider(ABC):
    def complete(self, system_prompt: str, context_docs: list[str], question: str) -> GroundedAnalystResponse: ...
```

- **Viva default (must work offline): Ollama** with `llama3.2:3b` (`LLM_PROVIDER=ollama`).
- **Development optional: Groq** with `llama-3.3-70b-versatile` when a key is available. **Do not cite unverified accuracy percentages** (no “98% grounding”).
- **Optional fallback: OpenAI** `gpt-4o-mini` if a key exists.
- **Configuration:** `LLM_PROVIDER=ollama|groq|openai` in `.env`. Viva demo uses Ollama.

What gets embedded (critical design decision): Agent-generated **output summaries**, not raw posts. Each agent produces a 2–4 sentence summary per analysis run, which is embedded and stored. Examples: - B: "*Between Sep 1–*

3, anxiety was dominant (42%) in monitored channels, peaking after the ISRO announcement post." - D: "*Topic 'satellite launch' shows velocity +340% over 48h, driven by 892 posts across 6 channels.*" - E: "*Cluster of 23 accounts exhibits high posting synchrony — anomaly score 0.78 — active primarily during 18:00–20:00 IST.*"

Context document provenance wrapper (temporal anchoring): Agent summaries passed to the LLM must not be bare text. To prevent temporal hallucinations and enable the model to reason accurately about timelines, the retriever wraps each retrieved chunk with standardized metadata:

```
[DOC_{i}] [Component: {source_component} | Window: {window_start} to {window_end} UTC | Ref: {source_record_id}]:
{content_text}
```

Grounded retrieval pipeline (one retrieve + optional one LLM pass): Do not claim the system is hallucination-free or that it implements published CRAG/Self-RAG papers. The pipeline is: retrieve summaries → refuse if far → one structured generation → log. To keep latency low on a student laptop: 1. **Tier 0 Hard Mathematical Cutoff (`< 0.65`):** Cosine distance search uses pgvector's `<=>` operator. If no document meets `< 0.65`, the backend short-circuits immediately without calling the LLM (`insufficient_data = True`): `sql SELECT id, source_component, content_text, window_start, window_end, (embedding <=> :query_vector) AS distance FROM rag_documents WHERE (embedding <=> :query_vector) < 0.65 -- Cosine distance cutoff ORDER BY distance ASC LIMIT 5;` 2. **Tier 1 Temporal / relevance filter:** Evaluates retrieved chunks against the query timeframe using document `window_start` / `window_end`. If chunks are anachronistic, return a deterministic fallback rather than interpolating. 3. **Tier 2 Single structured generation (Pydantic Schema):** One LLM call, not a multi-round agent loop: `python class GroundedAnalystResponse(BaseModel): relevance_grade: Literal["CORRECT", "AMBIGUOUS", "INSUFFICIENT"] answer: str cited_document_ids: list[str] is_grounded: bool confidence_rating: float = Field(ge=0.0, le=1.0)` If `relevance_grade == "INSUFFICIENT"` or `is_grounded == False`, the interface displays uncertainty flags. This reduces unsupported claims; it does not make the LLM hallucination-free.

Citation tracking & Auditability: - Every embedded document has a `source_id` and `source_table` - LLM cites `[DOC_1]`, `[DOC_2]` etc. - Backend parses citation tags and maps them to real database records - Non-bypassable audit logging into `rag_query_log` before returning HTTP payload

5.6 Frontend Layer & UI/UX Design System

Tool / Skill	Purpose
`React 18` + `Vite`	Application framework
`Tailwind CSS`	Styling & utility classes
`Recharts`	Sentiment timeline, trend velocity charts
`react-force-graph` / `Cytoscape.js`	Interactive network interaction graph
`React Query`	API data fetching and caching
`DESIGN.md` (awesome-design-md)	**Design System Ground Truth**: Linear Dark / Console tokens (#090d16 canvas, #0f172a cards, 1px #1e293b borders)
`Web Design Guidelines` (Vercel)	**Quality Gate & Linter**: Audits WCAG AA contrast, 8pt spacing grid, responsive reflow, and table overflow
`Taste Skill` (Leonxlnx)	**Anti-Slop Guardrail**: Enforces visual density (8/10), low motion (2/10), eliminates generic 3-column marketing templates
`Image-to-Code Skill` (Leonxlnx)	**Mockup Transpiler**: Converts operational layout references into pixel-perfect component shells

Locked Design Language Triad: 1. **Bento Grid (Structural Layout Blueprint):** Modular, asymmetric, responsive rectangular cells that cleanly partition analytical agents (A–E) onto a unified tactical operations canvas without fragmented page hopping. 2. **Dark Utilitarian Minimalism (Surface & Palette):** High data-ink ratio inspired by Linear Dark / Datadog. Deep canvas (`#090d16`), card surfaces (`#0f172a`), 1px solid borders (`#1e293b`), monospace metrics (`JetBrains Mono` or `Consolas`), and zero decorative marketing card slop. 3. **Spatial UI (Interactive Depth & HUD Elements):** Z-index elevation layering, contextual floating inspection drawers (for RAG citations and graph node inspections), and tactical HUD telemetry badges with subtle frosted glass accents (`backdrop-blur-md`).

Required dashboard views (7 core views):

1. **Overview/Home** — cross-component Bento summary for selected time range + instant "Run Analysis Now" trigger
2. **Sentiment & Emotion (B)** — stacked emotion timeline, stance breakdown, sarcasm-flagged posts shown separately with "lower confidence" label
3. **Trend Explorer (D)** — ranked topic list by velocity/acceleration, drill-down to contributing posts
4. **Demographics (C)** — **Audience aggregates (inferred)**; below-threshold clusters visibly greyed out with explanation tooltip; label is not census demographics
5. **Network Graph (E)** — interactive force graph, **betweenness-centrality-based** node sizing, anomaly nodes visually distinct, legend reads "anomaly signal — not confirmed"; graph capped at top 500 nodes
6. **Data Sources (A)** — which channels are monitored, ingestion stats, known dataset limitations
7. **Query Panel (RAG)** — chat-style interface; answer shown alongside source records; all queries logged to `rag_query_log`

React Router Client Route Map: | Route | Component | Description | |---|---|---| | `/` | `Overview.jsx` | Cross-component operational summary & KPIs | | `/sentiment` | `SentimentTimeline.jsx` | Component B: Emotion timeline, stance, sarcasm | | `/trends` | `TrendExplorer.jsx` | Component D: Ranked velocity & topic drilldown | | `/demographics` | `Demographics.jsx` | Component C: Aggregate demographic distributions | | `/network` |

`NetworkGraph.jsx` | Component E: Force-directed graph & CIB signals | | `/datasources` | `DataSources.jsx` | Component A: Channel health & ingestion stats | | `/query` | `QueryPanel.jsx` | RAG natural language inquiry interface |

Overview aggregation: `Overview.jsx` queries `/api/overview/summary` (served by `orchestrator/aggregator.py`), aggregating high-level metrics across all active agent runs into a single responsive payload.

UX principle: Every inference-based chart must visibly communicate confidence/uncertainty. Design for honesty and legibility over visual flash. Strict 150ms opacity transitions; zero bouncing physics or distracting slide animations.

5.7 DevOps / Reproducibility

Tool	Purpose
`Docker Compose`	Single-command system spinup
`.env.example`	Config template (no real secrets ever committed)
`pytest`	Backend endpoint tests + key ML pipeline function tests
`ARCHITECTURE.md`	Living architecture document, updated each phase

---

6. Data Schema — Definitive

This section is the **canonical schema**. `CURSOR_BRIEF.md` must not define a conflicting slim copy; implementers follow the tables below.

6.1 Raw Ingestion Tables (Component A — immutable after write)

`raw_posts`

```
-- Component A: Primary ingestion table. Immutable after insert.
id                  BIGSERIAL PRIMARY KEY
platform            VARCHAR(32) NOT NULL DEFAULT 'telegram'
platform_post_id    VARCHAR(255) NOT NULL          -- platform's native ID (used for deduplication)
channel_id          VARCHAR(255) NOT NULL          -- channel identifier
author_id           VARCHAR(255)                   -- platform author identifier (may be null for broadcast)
content_text        TEXT                           -- raw post text
content_language    VARCHAR(16)                    -- detected language code (ISO 639-1)
media_urls          JSONB                          -- array of attached media URLs
reply_to_id         VARCHAR(255)                   -- parent post if reply/thread
forward_from_id     VARCHAR(255)                   -- original source if forwarded
view_count          INTEGER                        -- at time of ingestion
reaction_counts     JSONB                          -- {"👍": 42, "❤️": 12, ...}
ingested_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
posted_at           TIMESTAMPTZ NOT NULL           -- original post timestamp (Component A timeline anchor)
ingestion_run_id    UUID REFERENCES ingestion_runs(id)
processing_status   VARCHAR(32) DEFAULT 'pending'   -- 'pending', 'processing', 'done', 'error' (Component B only)
UNIQUE(platform, platform_post_id)
```

raw_authors

```
-- Component A: Author metadata at time of ingestion. Append new row on change, never update.
-- Note: Analytical queries must use:
-- SELECT DISTINCT ON (platform, platform_author_id) * FROM raw_authors ORDER BY platform, platform_author_id, sna
-- to prevent duplicate counting across historical snapshots.
id                  BIGSERIAL PRIMARY KEY
platform            VARCHAR(32) NOT NULL
platform_author_id  VARCHAR(255) NOT NULL
display_name        VARCHAR(512)
bio_text            TEXT                           -- used by Component C for demographic inference
follower_count      INTEGER
account_created_at  TIMESTAMPTZ                    -- used by Component E for CIB age-vs-volume signal
is_verified         BOOLEAN
snapshot_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
ingestion_run_id    UUID REFERENCES ingestion_runs(id)
```

ingestion_runs

```
-- Component A: Tracks each scheduled ingestion batch.
id                  UUID PRIMARY KEY DEFAULT gen_random_uuid()
started_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
completed_at        TIMESTAMPTZ
platform            VARCHAR(32) NOT NULL
channel_ids         JSONB                          -- channels targeted in this run
posts_collected     INTEGER DEFAULT 0
status              VARCHAR(32) DEFAULT 'running'  -- 'running', 'complete', 'error'
error_message       TEXT
```

monitored_channels

```
-- Component A: Single source of truth for which channels are monitored.
id                  BIGSERIAL PRIMARY KEY
platform            VARCHAR(32) NOT NULL
channel_identifier  VARCHAR(255) NOT NULL          -- @username or channel ID
display_name        VARCHAR(512)
topic_tags          JSONB                          -- ["space", "ISRO", "technology"]
is_active           BOOLEAN DEFAULT TRUE
added_at            TIMESTAMPTZ NOT NULL DEFAULT NOW()
added_by            VARCHAR(255)                   -- team member who added this channel
notes               TEXT                           -- why this channel was selected
```

6.2 Derived Inference Tables (Components B–E — traceable to source)

sentiment_results_b

```
-- Component B output. Every row traces to a raw_posts row.
-- Single-model architecture: all text uses GoEmotions (28 classes) for emotion;
-- cardiffnlp/twitter-xlm-roberta-base-sentiment for sentiment.
id                  BIGSERIAL PRIMARY KEY
raw_post_id         BIGINT NOT NULL REFERENCES raw_posts(id)
emotion_model       VARCHAR(128) NOT NULL          -- 'SamLowe/roberta-base-go_emotions'
valence_model       VARCHAR(128) NOT NULL          -- 'cardiffnlp/twitter-xlm-roberta-base-sentiment'

-- CORE EMOTIONS: 7-class subset (all posts)
-- All values 0.0–1.0
emotion_joy         FLOAT
emotion_anger       FLOAT
emotion_sadness     FLOAT
emotion_disgust     FLOAT
emotion_surprise    FLOAT
emotion_neutral     FLOAT
emotion_fear        FLOAT

-- EXTENDED EMOTIONS: GoEmotions additional classes
emotion_anxiety     FLOAT
emotion_excitement  FLOAT
emotion_admiration  FLOAT
emotion_gratitude   FLOAT
emotion_curiosity   FLOAT
emotion_approval    FLOAT
emotion_annoyance   FLOAT
-- (remaining GoEmotions categories stored as JSONB to avoid column explosion)
emotion_extended    JSONB                          -- full GoEmotions output

-- Dominant emotion and confidence
dominant_emotion    VARCHAR(64)                    -- highest-scoring label from GoEmotions
emotion_confidence  FLOAT

-- Stance detection (separate model, language-agnostic)
stance              VARCHAR(32)                    -- 'supportive', 'against', 'neutral', 'unclear'
stance_confidence   FLOAT

-- Sarcasm (heuristic signal, not a confirmed label)
sarcasm_flag        BOOLEAN DEFAULT FALSE
sarcasm_note        TEXT

-- Overall valence
valence             VARCHAR(16)                    -- 'positive', 'negative', 'neutral'
valence_confidence  FLOAT

-- Metadata
```

```
analyzed_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
analysis_batch_id   UUID
```

trend_topics_d

```
-- Component D output. Topic snapshots per analysis run.
id                  BIGSERIAL PRIMARY KEY
topic_label         VARCHAR(512) NOT NULL           -- human-readable topic description
topic_keywords      JSONB NOT NULL                  -- ["satellite", "launch", "ISRO"]
window_start        TIMESTAMPTZ NOT NULL            -- time window this snapshot covers
window_end          TIMESTAMPTZ NOT NULL
post_count          INTEGER NOT NULL                -- posts assigned to this topic in window
velocity            FLOAT NOT NULL                  -- posts/hour rate
acceleration        FLOAT NOT NULL                  -- change in velocity vs. previous window
rank                INTEGER                         -- rank by velocity in this window
bertopic_model_id   VARCHAR(128)                    -- model version used
sample_post_ids     JSONB                           -- array of representative raw_post_ids
analyzed_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
analysis_batch_id   UUID
```

demographic_aggregates_c

```
-- Component C output. Aggregate only — never individual-level data.
id                  BIGSERIAL PRIMARY KEY
cluster_label       VARCHAR(255) NOT NULL           -- descriptive cluster name
window_start        TIMESTAMPTZ NOT NULL
window_end          TIMESTAMPTZ NOT NULL
account_count       INTEGER NOT NULL                -- number of accounts in this cluster
below_threshold     BOOLEAN NOT NULL DEFAULT FALSE -- TRUE if account_count < MIN_CLUSTER_SIZE
-- Inferred aggregate characteristics (all are distributions, not individual labels)
age_bracket_dist    JSONB                           -- {"18-24": 0.32, "25-34": 0.45, ...}
geography_dist      JSONB                           -- {"IN-KA": 0.28, "IN-MH": 0.21, ...}
language_dist       JSONB                           -- {"en": 0.61, "hi": 0.29, ...}
interest_dist       JSONB                           -- {"technology": 0.55, "space": 0.71, ...}
-- Confidence
inference_confidence FLOAT                          -- aggregate confidence across all inferences
analysis_notes      TEXT                            -- any caveats about this cluster
analyzed_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
analysis_batch_id   UUID
```

network_nodes_e

```
-- Component E output. One row per entity (channel or author) per analysis window.
id                  BIGSERIAL PRIMARY KEY
entity_id           VARCHAR(255) NOT NULL          -- channel identifier or platform_author_id
entity_type         VARCHAR(32) NOT NULL           -- 'channel' or 'author'
label               VARCHAR(512) NOT NULL          -- channel display title, or pseudonymized identifier for flagg
window_start        TIMESTAMPTZ NOT NULL
window_end          TIMESTAMPTZ NOT NULL
-- Centrality metrics
degree_centrality   FLOAT
betweenness_centrality FLOAT                       -- why: identifies bridging accounts between communities
eigenvector_centrality FLOAT
-- Community
community_id        INTEGER                        -- Louvain community assignment
-- CIB anomaly signals (heuristic — not verdicts)
anomaly_score       FLOAT                          -- composite score 0.0–1.0; higher = more anomalous
posting_synchrony_signal FLOAT                     -- 0.0–1.0; high = posts cluster tightly in time with others
age_volume_signal   FLOAT                          -- 0.0–1.0; high = new account with disproportionate volume
duplicate_content_signal FLOAT                     -- 0.0–1.0; high = many near-duplicate posts
coordination_signal BOOLEAN DEFAULT FALSE          -- TRUE if anomaly_score > threshold (default: 0.7)
analyzed_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
analysis_batch_id   UUID
```

network_edges_e

```
-- Component E output. Directed interactions between entities (channels or authors).
id                  BIGSERIAL PRIMARY KEY
source_entity_id    VARCHAR(255) NOT NULL
target_entity_id    VARCHAR(255) NOT NULL
interaction_type    VARCHAR(64) NOT NULL           -- 'reply', 'forward', 'mention'
weight              FLOAT NOT NULL DEFAULT 1.0     -- interaction frequency-based weight
window_start        TIMESTAMPTZ NOT NULL
window_end          TIMESTAMPTZ NOT NULL
analyzed_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
analysis_batch_id   UUID
```

6.3 RAG / Embedding Tables

rag_documents

```
-- RAG layer: Stores agent-generated summaries for embedding and retrieval.
id                  UUID PRIMARY KEY DEFAULT gen_random_uuid()
source_component    VARCHAR(8) NOT NULL            -- 'B', 'C', 'D', 'E'
source_table        VARCHAR(128) NOT NULL           -- originating derived table
source_record_id    TEXT NOT NULL                  -- ID of the originating record
content_text        TEXT NOT NULL                  -- the 2–4 sentence summary to be embedded
embedding           vector(384)                    -- paraphrase-multilingual-MiniLM-L12-v2 produces 384-dim vecto
window_start        TIMESTAMPTZ
window_end          TIMESTAMPTZ
created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

rag_query_log

```
-- RAG layer: Mandatory audit log of all queries and responses.
-- Every query must be logged before the response is returned to the frontend.
-- This table is part of the analytical audit trail; rows must never be deleted.
id                  UUID PRIMARY KEY DEFAULT gen_random_uuid()
question_text       TEXT NOT NULL
retrieved_doc_ids   JSONB NOT NULL                 -- array of rag_documents.id used
relevance_grade     VARCHAR(16) NOT NULL           -- 'CORRECT', 'AMBIGUOUS', 'INSUFFICIENT'
llm_provider        VARCHAR(64) NOT NULL           -- 'groq', 'ollama', 'openai'
answer_text         TEXT NOT NULL
cited_source_ids    JSONB                          -- parsed citation tags mapped to real DB record IDs
confidence_rating   FLOAT DEFAULT 1.0              -- self-reflective confidence score (0.0 to 1.0)
is_grounded         BOOLEAN DEFAULT TRUE           -- self-reflective assertion flag
evaluation_notes    TEXT                           -- optional analyst feedback
queried_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

6.3a Database Performance Indexes & Scaling Constraints

To prevent table scans across large post volumes and optimize Celery worker queries:

```
-- Component B row-level worker optimization (pending post queue)
CREATE INDEX idx_raw_posts_pending ON raw_posts (id) WHERE processing_status = 'pending';

-- Components C, D, E window-level range query optimization
CREATE INDEX idx_raw_posts_posted_at ON raw_posts (posted_at DESC);

-- Traceability joins (derived -> raw)
CREATE INDEX idx_sentiment_results_raw_post ON sentiment_results_b (raw_post_id);

-- Vector similarity search acceleration (pgvector HNSW index, cosine distance)
CREATE INDEX idx_rag_documents_hnsw ON rag_documents USING hnsw (embedding vector_cosine_ops);
```

6.4 API Response Contracts (Frontend ↔ Backend)

These are Pydantic schemas — the frontend builds against these from day one.

```
# GET /api/ingestion-a/stats
class IngestionStatsResponse(BaseModel):
    total_posts: int
    monitored_channel_count: int
    latest_ingested_at: str | None
    oldest_post_at: str | None
    active_runs: list[IngestionRunSummary]

class IngestionRunSummary(BaseModel):
    run_id: str
    started_at: str
    posts_collected: int
    status: str

# GET /api/overview/summary
class OverviewSummaryResponse(BaseModel):
    total_monitored_channels: int
    total_posts_ingested: int
    active_window_start: str
    active_window_end: str
    dominant_emotion: str
    top_trending_topic: str
    flagged_anomaly_nodes_count: int
    latest_ingestion_run_status: str

# GET /api/sentiment-b/timeline
class SentimentTimelineResponse(BaseModel):
    time_bucket: str                    # ISO timestamp, bucket granularity configurable
    emotion_breakdown: dict[str, float] # {"joy": 0.12, "anxiety": 0.42, ...}
    dominant_emotion: str
    stance_breakdown: dict[str, float]  # {"supportive": 0.31, "against": 0.45, ...}
    sarcasm_flagged_count: int
    post_count: int

# GET /api/trends-d/ranked
class TrendTopicResponse(BaseModel):
    topic_id: int
    label: str
    keywords: list[str]
    velocity: float
    acceleration: float
    post_count: int
    sample_posts: list[str]             # excerpt list

# GET /api/network-e/graph
# Note: edges returned are strictly edge-induced (source_id IN top_500 AND target_id IN top_500)
# to prevent react-force-graph canvas renderer crashes caused by dangling edges.
```

```
class NetworkGraphResponse(BaseModel):
    nodes: list[NetworkNode]
    edges: list[NetworkEdge]
    total_nodes: int
    total_edges: int
    capped_at: int | None               # null if full graph returned; 500 if capped for performance

class NetworkNode(BaseModel):
    id: str                             # entity_id
    entity_type: str                    # 'channel' or 'author'
    centrality: float                   # betweenness centrality
    community_id: int
    anomaly_score: float
    coordination_signal: bool
    label: str                          # public channel title, OR pseudonymized (e.g. 'Actor-CIB-042 [Masked]') f

class NetworkEdge(BaseModel):
    source: str                         # source entity_id
    target: str                         # target entity_id
    weight: float
    interaction_type: str               # 'forward', 'reply', 'mention'

# GET /api/demographics-c/aggregates
class DemographicClusterResponse(BaseModel):
    cluster_label: str
    account_count: int
    below_threshold: bool               # if True, UI greys this out
    age_bracket_dist: dict[str, float]
    geography_dist: dict[str, float]
    language_dist: dict[str, float]
    interest_dist: dict[str, float]
    inference_confidence: float

# POST /api/rag/query
class RAGQueryRequest(BaseModel):
    question: str

class RAGQueryResponse(BaseModel):
    answer: str
    sources: list[RAGSource]
    relevance_grade: str                # 'CORRECT', 'AMBIGUOUS', 'INSUFFICIENT'
    confidence_rating: float            # 0.0 to 1.0
    is_grounded: bool                   # True if claims are strictly grounded in sources
    llm_provider: str
    insufficient_data: bool             # True if Tier 0/1/2 returned the grounding-failure response

class RAGSource(BaseModel):
```

```
    doc_id: str
    source_component: str              # 'B', 'C', 'D', 'E'
    excerpt: str
    window_start: str
    window_end: str

# POST /api/rag/feedback
class RAGFeedbackRequest(BaseModel):
    query_id: str                      # references rag_query_log.id
    grounded: bool                     # analyst evaluation: true if answer is strictly backed by context
    evaluation_notes: str | None = None

class RAGFeedbackResponse(BaseModel):
    status: str                        # 'recorded'
    query_id: str
```

---

7. Evaluation Metrics — Per Component

Component B (Sentiment Agent)

English track (`SamLowe/roberta-base-go_emotions`): - Valence: run against **TweetEval Sentiment** (~60k tweets) — report accuracy and macro F1 - Emotion: run against **GoEmotions test split** — report macro F1 per class and overall - Stance: run against **SemEval 2016 Task 6** if time permits - Sarcasm: run against **TweetEval Irony** (~4.6k) + **SARC balanced** (~250k sample)

Evaluated alternatives (L3Cube-Pune — documented for report only): - L3Cube-Pune (Pune, India) publishes HingBERT (`l3cube-pune/hing-bert`), HindiBERT (`l3cube-pune/hindi-bert-v2`), and Hindi-Tweets-BERT (`l3cube-pune/hindi-tweets-bert`) — base language models pre-trained on 52.9M Hindi-English code-mixed sentences from Twitter. - These are base encoders, not sentiment classifiers. Using them would require fine-tuning a classification head on a labeled Hindi/Hinglish sentiment dataset — an engineering investment disproportionate to the project timeline. - The project report should include: "*We evaluated L3Cube-Pune's HingBERT and HindiBERT as potential encoders for code-mixed text. While these models offer native Hinglish pre-training on 52.9M social media sentences, they are base language models requiring task-specific fine-tuning. We selected the pre-fine-tuned multilingual twitter-xlm-roberta-base-sentiment (1.15M downloads, 198M tweet training set) to minimize engineering risk and ensure demo reliability.*"

Acknowledge: domain gap (benchmarks are Reddit/Twitter; deployment is Telegram)

Component C (Demographic Agent)

- **Heuristic Quality Validation (Manual Qualitative Spot-Check):**
- Because public ground truth does not exist for unlabelled Telegram audiences, the team conducts a structured qualitative audit of **20–30 public author profiles** sampled from active discussion threads whose bios, self-descriptions, or explicit post declarations contain verifiable demographic indicators (e.g., self-declared student/professional occupation, explicit regional/city markers, explicit age/generation cues).
- Annotate ground-truth attributes manually and compute heuristic concordance (precision) across:

1. Primary inferred language vs. actual bio/message text language.
2. Inferred geography/region vs. explicit location/timezone indicators.
3. Inferred interest tags vs. topical posting content.

- Report qualitative agreement percentage and failure modes (e.g., bio parsing false positives from ironic bios or quote snippets) in the written report and viva defense, validating inference quality rather than relying solely on privacy-filter survival rates.
- **Privacy & Coverage Metrics:**
- Cluster size distribution
- % of accounts/posts covered vs. filtered by minimum cluster guard ($k \ge 50$)
- Geographic and language coverage breadth across active discussion participants

Component D (Trend Agent)

- **Manual validation:** At end of each analysis period, check top 5 trending topics against news headlines for the same period. Report as "X of 5 top topics corresponded to verifiable real-world events"
- Report topic coherence (BERTopic built-in metric: c_v coherence score)

Component E (Network Agent)

Offline benchmark (corrected dataset description — resolves synopsis precision/recall claim): - Dataset: `ioa_users.csv`, sourced from Kaggle (`pookiewiggington/twitter-information-operations-classification`) — see Section 4.1a for full provenance - Ground truth: built-in `info_op` column (1 = coordinated IRA-linked account, 0 = organic) — no manual control set construction required - Signals benchmarked: - ✅ `posting_synchrony_signal` — derived from `tweet_time` per `userid` - ✅ `duplicate_content_signal` — derived from `BoW` column via TF-IDF cosine similarity - ⚠️ `age_volume_signal` — **excluded from this benchmark** (account creation date absent from dataset; stated explicitly in report) - Aggregate tweet rows to user level before evaluation: `userid → { tweet_times, bow_merged, info_op }` - Compute and report: **precision, recall, F1** at default threshold (0.7); also report precision-recall curve across 0.0–1.0. These numbers belong in the **written report**, not as a dashboard KPI that looks like a conviction rate. - Use a **documented user-level sample** of the 11M-row file if full-file runtime is impractical; state sample size and sampling method. - Script location: `benchmarks/cib_ioa/evaluate.py` — standalone, not part of the live pipeline - Acknowledge in report: domain shift (dataset is Twitter, Russian-operation-specific; live system monitors Telegram, India-focused) — results are indicative, not directly generalizable; cite cross-country generalizability research (Section 4.1a) to support applicability

Live system reporting (on Telegram data): - Anomaly score distribution across all analyzed accounts - Descriptive characterization of accounts where `coordination_signal = TRUE` (posting frequency, content similarity patterns) - UI legend and report both state: "anomaly signal — not confirmed coordination; two signals validated against an offline benchmark dataset (Twitter domain, see Section 4.1a for provenance)"

RAG Layer

- 10 hand-crafted test questions with expected answers based on actual collected data
- Run before viva, report:
- % of answers correctly grounded in cited sources
- % of cited sources actually containing the claimed information
- 1–2 documented failure cases with analysis

---

8. Build Order — Enforced Phase Plan

Do not start a phase before its predecessor is producing real output.

Phase 1 — Foundation (Weeks 1–4)

Owner: P1 (primary), all review - [ ] Lock `config/channels.yaml` — team selects 30 Telegram channels, documents selection criteria and staggered 30s polling intervals - [ ] Implement `BaseIngestionAdapter` interface - [ ] Implement `TelegramIngestionAdapter` (Pyrofork, public channels, rate-limit handling) - [ ] Define and migrate full database schema (all tables above; initial migration executes `CREATE EXTENSION IF NOT EXISTS vector;`) - [ ] Celery + Redis task queue setup - [ ] Celery Beat 15-minute scheduler wired to ingestion - [ ] Real data flowing into `raw_posts` and `raw_authors` with correct timestamps - [ ] Basic FastAPI health endpoint - [ ] Docker Compose running (Postgres + pgvector + Redis + backend skeleton) - [ ] `ARCHITECTURE.md` initial version committed

Checkpoint 1 deliverable: Live Telegram ingestion running, real posts in database, schema migrated, Docker Compose functional.

Phase 2 — Core NLP (Weeks 5–8)

Owners: P2 (B + D), P3 (orchestrator skeleton) - [ ] Implement `SentimentAgent` (B): emotion + valence + stance + sarcasm flag, writing to `sentiment_results_b` - [ ] Implement `TrendAgent` (D): BERTopic pipeline, velocity/acceleration calculation, writing to `trend_topics_d` - [ ] Implement orchestrator row-level runner for B: queries `raw_posts WHERE processing_status = 'pending'`, fires `SentimentAgent` per row, marks rows `done` — runs every `COMPONENT_B_INTERVAL_MINUTES` (default 15 min) - [ ] Register `TrendAgent` (D) as a separate Celery Beat schedule (`COMPONENT_D_INTERVAL_MINUTES`, default 60 min) — queries `raw_posts` by 48h timestamp range, no `processing_status` involvement - [ ] FastAPI endpoints for sentiment timeline and trend ranking - [ ] Unit tests for B and D pipeline functions

Checkpoint 2 deliverable: Sentiment and trend outputs for real Telegram data, accessible via API, orchestrator running scheduled batches.

Phase 3 — Remaining Analytics (Weeks 9–11)

Owners: P3 (E), P4 (C) - [ ] Implement `DemographicAgent` (C): bio-text inference, minimum cluster size guard, writing to `demographic_aggregates_c` - [ ] Implement `NetworkAgent` (E): graph construction, centrality, Louvain communities, CIB heuristic signals, writing to `network_nodes_e` and `network_edges_e` - [ ] Register `DemographicAgent` (C) as independent Celery Beat schedule (`COMPONENT_C_INTERVAL_MINUTES`, default 60 min) — queries by timestamp range - [ ] Register `NetworkAgent` (E) as independent Celery Beat schedule (`COMPONENT_E_INTERVAL_MINUTES`, default 120 min) — queries by configurable window - [ ] Verify all four schedules are independently configurable via `.env` and do not share state via `processing_status` - [ ] FastAPI endpoints for demographics and network graph - [ ] Verify network graph is capped at a sensible node count (top 500 by centrality) for frontend performance - [ ] Unit tests for C and E pipeline functions

Checkpoint 3 deliverable: All five agents producing output, all API endpoints returning real data.

Phase 4 — Integration + UI (Weeks 11–13)

Owners: P4 (RAG + backend polish), P1 + frontend across team - [ ] Embedding pipeline: agent summaries → `sentence-transformers` → `pgvector` - [ ] RAG query endpoint: retrieval + grounded LLM answer + citation tracking - [ ] Ollama setup as **viva default** + optional Groq/OpenAI, switchable via `.env` - [ ] All 7 frontend dashboard views listed in Section 5.6 operational and connected to FastAPI backend - [ ] Network graph with performance cap + anomaly node distinction - [ ] HTTP Basic Auth middleware - [ ] HTTP polling for dashboard refresh (**no WebSockets**) - [ ] API contract finalization — all Pydantic schemas locked and matched by frontend

---

Phase 5 — Polish (Weeks 13–15)

All team members - [ ] Run evaluation benchmarks for B (SemEval), D (manual validation) - [ ] **Component E offline CIB benchmark:** - [ ] Confirm `ioa_users.csv` in `benchmarks/cib_ioa/` (already downloaded — no additional access needed) - [ ] Aggregate to user level: `benchmarks/cib_ioa/aggregate_users.py` - [ ] Run `benchmarks/cib_ioa/evaluate.py` — outputs precision, recall, F1, and precision-recall curve for `posting_synchrony_signal` and `duplicate_content_signal` - [ ] Write up results: include threshold sensitivity table, note that `age_volume_signal` was not benchmarked (no account creation date in dataset), acknowledge Twitter→Telegram domain shift and Russia→India geographic mismatch, cite cross-country generalizability research (Section 4.1a) - [ ] Run RAG evaluation (10 test questions) - [ ] `ARCHITECTURE.md` complete and current - [ ] Generate plain-language summaries for each component (3–5 sentences, for report and team briefing) - [ ] Report writing — each person writes the section for their component - [ ] Viva prep — every teammate reads and can explain every other component

---

9. Non-Negotiable Ethical / Technical Constraints

1. **Public data only** — no login-walled scraping, no private group access, no impersonation, no credential sharing
2. **Append-only ingestion** — raw records immutable after write; every derived field references its source `raw_post_id`; historical records collected in Postgres remain persistent and immutable during live operations
3. **Demographic outputs aggregate-only** — enforced by `MIN_CLUSTER_SIZE` guard in code (not just policy). Report figure is 50; demo `.env` may use a lower k because seed volume is small. Sub-threshold clusters are greyed out, never hidden, never padded with fake users.
4. **Confidence/uncertainty propagates end-to-end** — from backend response schema (`_confidence` fields) through to UI display; no chart hides model uncertainty
5. **CIB/anomaly detection is a signal, never a verdict** — naming in code (`anomaly_score`, `coordination_signal`) and UI labels ("anomaly signal — not confirmed") must be consistent throughout; never write "bot", "coordinated account", or similar definitive language in outputs
6. **No individual account attribution in outputs** — accounts flagged by Component E must never be named, displayed by handle, or identified in any report or export; only aggregate patterns and anonymised cluster characteristics (posting time distributions, content similarity scores, community structure) are reportable externally; individual `platform_author_id` values remain internal to the database only. This constraint exists because false positives misattribute coordinated behaviour to legitimate actors — an error with real consequences when the tool is adjacent to a government problem statement.

9.1 Regulatory & Compliance Alignment (CERT-In Directions & DPDP Act 2023)

While the platform is an academic research prototype / PoC exempt from direct enterprise reporting obligations (which legally apply to commercial intermediaries, cloud service providers, and data centers under Section 70B of the IT Act), the system proactively incorporates key Indian cybersecurity and data privacy principles to ensure defensibility during institutional review and viva defense:

1. **Lawful OSINT Boundary:** Ingestion operates exclusively on publicly accessible Telegram broadcast channels via Pyrofork MTProto. No private channels, private groups, or direct messages (DMs) are ever accessed, strictly honoring lawful open-source intelligence boundaries.
2. **DPDP Act 2023 *principles* via k-anonymity:** Component C enforces data minimization by flagging cohorts smaller than `MIN_CLUSTER_SIZE` (`below_threshold: true`). **Do not claim the prototype is DPDP-compliant.** State that k-anonymity and audit logging follow DPDP/CERT-In *principles*. Report default k is 50.
3. **CERT-In Timestamp Synchronization:** Aligning with CERT-In's NTP clock synchronization directive, all temporal ingestion and velocity calculations are strictly tied to UTC `TIMESTAMPTZ` (`posted_at` and `ingested_at`), guaranteeing forensically verifiable chronological integrity.
4. **CERT-In Analyst Auditability & Non-Repudiation:** Every analyst query to the RAG layer is recorded with timestamp, query string, and retrieved document references in `rag_query_log` to prevent untraceable intelligence misuse.
5. **Advisory Anomaly Guardrail:** CIB heuristics output advisory probability scores rather than definitive verdicts, eliminating the legal/ethical risk of falsely defaming legitimate accounts.
6. **Data Lifecycle & Post-Viva Scrub:** Personal post contents and author bios are wiped from local development instances post-viva via a simple committed SQL scrub script.

---

10. Decisions Log

Decisions made before Phase 1 begins. Each teammate must read this.

Decision	Choice	Reason
X/Twitter (live)	Removed from live pipeline	ToS violation for scraping; research archive requires application outside timeline
Offline CIB benchmark dataset provenance	Corrected: Kaggle `pookiewiggington/twitter-information-operations-classification`, NOT Twitter's raw official archive	Verified via manual inspection (both info_op=0 and info_op=1 rows present); this is a third-party combination of confirmed IRA accounts + separately-built organic control set — official archive releases are positive-only and never natively include organic labels
CIB validation approach	Kaggle dataset with built-in `info_op` label	Built-in ground truth for both classes; no manual control set needed; verified at 11M+ row scale; age_volume_signal excluded (no account creation date)
Cross-country generalizability	Explicit justification required in report	Benchmark is Russian-operation data validating an India-focused system; cite published research showing content-based coordination features generalize across countries/platforms
Orchestrator trigger	Celery Beat, 15-min batch	Simpler than real-time event-driven; sufficient for intelligence analysis latency
LLM for RAG	**Viva default: Ollama `llama3.2:3b`.** Optional Groq/OpenAI for development. No unverified accuracy percentages.	Offline viva must not depend on cloud APIs
What to embed	Agent output summaries (2–4 sentences), not raw posts	Retrieval quality > embedding volume for this use case
Embedding model	`paraphrase-multilingual-MiniLM-L12-v2` (384-dim)	Multilingual (50+ languages) required for Hindi Telegram content; same dimension as English-only all-MiniLM-L6-v2, similar speed
Min cluster size (C)	Report k=50; demo k configurable (e.g. 15)	Ethics figure stays 50; 30 channels provide adequate commenter volume for k≥50 clusters
Network graph cap	Top 500 nodes by **betweenness** centrality	Betweenness surfaces coordination brokers (accounts bridging communities); degree centrality alone surfaces loudest accounts, not most structurally significant
Centrality primary sort	Betweenness centrality	Highest-value signal for CIB detection; degree and eigenvector computed and stored but not used for the cap
CIB signal count (3 vs 4)	Pruned to 3 core signals	forwarding_synchrony_signal cut to simplify schema and avoid missing forward timestamp dependencies; anomaly score uses posting_synchrony, age_volume, and duplicate_content
RAG query logging	Mandatory, non-bypassable	Analytical audit trail; every query+response logged before frontend response returned; rows never deleted
posted_at for D/E time windows	`posted_at` not `ingested_at`	`ingested_at` measures polling lag; all temporal analysis must use actual message timestamps
Hindi/Devanagari pipeline	Dropped — not needed	Target channels produce English + Hinglish only; evaluated L3Cube-Pune's HingBERT/HindiBERT but excluded as base models requiring fine-tuning; multilingual xlm-roberta provides adequate coverage

Decision	Choice	Reason
Stance detection model	Zero-shot NLI; **runtime DistilBART-MNLI**; BART-large optional	Laptop-viable default; heavier model only if hardware allows
Sarcasm heuristics	Formalized rule-based composite	Emoji incongruity, intensifier contradiction, and punctuation clustering in `sarcasm.py`
Demographic methodology	Dual-tier observable sampling	Channel macro metadata + linked discussion thread participant sampling; respects Telegram private member lists
BERTopic calibration	`min_topic_size=3` + `reduce_outliers`	Prevents HDBSCAN noise collapse on low-volume demo data; guarantees zero dropped posts
Louvain graph projection	`G.to_undirected()`	Projects directed interaction graph to undirected to satisfy python-louvain mathematical requirement
Composite anomaly math	Simple unweighted arithmetic mean	$(s_1 + s_2 + s_3) / N_{\text{available}}$ over non-null signals; simple, intuitive, and avoids uncalibrated heuristic weights on unannotated data
RAG distance cutoff	`< 0.65` cosine distance short-circuit	Enforces grounding at SQL layer; immediate refusal without LLM call if no relevant document passes cutoff
Grounded RAG	Retrieve summaries → cosine < 0.65 or refuse → one structured LLM pass → log	Do not claim published CRAG/Self-RAG implementations or “hallucination-free” output
Author snapshot query	`DISTINCT ON (platform, platform_author_id)`	Prevents duplicate author row joins across historical snapshots
Demo seed script	Historical replay mode (`--offline`)	Replays 1,000 real posts from `seed_demo_data.sql` ; ensures viva demo operates offline if campus Wi-Fi drops
Viva vs live ingest	Offline seed + Ollama is the exam path; live Pyrofork is optional at viva	Campus firewalls and FloodWait must not fail the demonstration
Run Analysis Now	Small B batch + capped C/D/E window and/or read derived tables; ~30s	Not a full recompute of all models on 1,000 posts
Embedding model reuse	One MiniLM for BERTopic and RAG	Avoid loading two sentence-transformer models
BERTopic volume cap	Max 500 docs per run (configurable)	Prevents worker OOM on “Run Analysis Now”
Alembic vector extension	`CREATE EXTENSION` in initial migration	Ensures pgvector extension is initialized before `rag_documents` table definition
Model singletons	Celery worker pre-warming	Loads model pipelines once on worker init (`celeryd_init`); prevents 15s cold starts and OOM crashes
Database indexes	Partial pending + HNSW vector	Partial index on pending rows and HNSW on pgvector ensure sub-second queries at 100k+ rows
Report export (View 8)	Pruned from scope	Formal PDF/HTML generation engine deferred to post-viva; analysts interact with 7 dashboard views and query RAG Assistant
.session file	First-class secret, outside repo	MTProto session = logged-in account; `*.session` in .gitignore; path via SESSION_FILE_PATH env var

Decision	Choice	Reason
Celery session worker	Concurrency=1 for ingestion	SQLite session file deadlocks if multiple worker processes open it concurrently
Deletion tracking	Deferred to future hardening	Active reconciliation sweeps deferred; batch pipeline ingests forward-moving feed; collected posts remain immutable in Postgres
Evidentiary hash	Pruned from scope	SHA-256 hashing on raw_posts deferred; academic PoC relies on Postgres append-only relational immutability and platform constraints
Network entity typing	Channels & authors as entities	Accommodates Telegram channel broadcasts (where author_id is null) without schema failures
Flagged actor privacy	Pseudonymization in UI/export	Individual users with coordination_signal=True masked as Actor-CIB-XXX to prevent false-positive attribution
Edge-induced graph cap	source IN top_500 AND target IN top_500	Prevents react-force-graph canvas crashes from dangling edges
RAG context wrapper	Standardized metadata header	Injects Component, Window, Ref into context chunks to prevent temporal hallucinations
Auth	HTTP Basic Auth (FastAPI middleware)	Sufficient for demo; document as future work for production-grade auth
No LangChain/LlamaIndex	Raw SQL + sentence-transformers + one LLM call	Simpler = more defensible in viva; every line of the RAG pipeline is team-owned
Heavy agent frameworks	Not used (no LangGraph, CrewAI)	Orchestrator is batch coordinator, not autonomous multi-step LLM agent
Reddit/YouTube	Report-only future work	**Do not scaffold** `reddit.py` / `youtube.py`
Data retention	Semester-end disposal step defined	Raw PII (raw_authors.bio_text, raw_posts.content_text) to be dropped after viva; documented in ARCHITECTURE.md; institutional ethics sign-off to be confirmed before Phase 1

---

11. Repository Structure (Preview)

```
socmint-framework/
├── ARCHITECTURE.md              # Living architecture doc — update every phase
├── README.md                    # Setup + run instructions
├── docker-compose.yml
├── .env.example
├── config/
│   └── channels.yaml            # Team-curated Telegram channel list (source of truth)
├── scripts/
│   └── seed_demo_data.sql       # 1,000 pre-packaged real historical posts for offline viva demo
│
├── ingestion_a/                 # Component A — Ingestion Agent
│   ├── adapters/
│   │   ├── base.py              # BaseIngestionAdapter interface
│   │   └── telegram.py          # TelegramIngestionAdapter (Pyrofork)
│   │                            # Do not scaffold reddit.py / youtube.py
│   ├── backfill.py              # Historical data replay / offline demo seed script
│   ├── import_exported.py       # Telegram Desktop JSON export → raw_posts (--import-json)
│   ├── tasks.py                 # Celery tasks
│   ├── scheduler.py             # Celery Beat schedule definitions
│   └── models.py                # Pydantic models for raw ingested data
│
├── storage/                     # Storage layer (serves Component A; used by all)
│   ├── database.py              # SQLAlchemy engine, session management
│   ├── migrations/              # Alembic migrations
│   └── models/                  # SQLAlchemy ORM models (one file per table group)
│       ├── raw.py               # raw_posts, raw_authors, ingestion_runs, monitored_channels
│       ├── derived_b.py         # sentiment_results_b
│       ├── derived_c.py         # demographic_aggregates_c
│       ├── derived_d.py         # trend_topics_d
│       ├── derived_e.py         # network_nodes_e, network_edges_e
│       └── rag.py               # rag_documents, rag_query_log
│
├── sentiment_b/                 # Component B — Sentiment Agent
│   ├── agent.py                 # SentimentAgent class (independently invocable)
│   ├── emotion.py               # Emotion detection pipeline
│   ├── stance.py                # Stance detection pipeline (separate from emotion)
│   ├── sarcasm.py               # Sarcasm heuristic flagging
│   └── summary.py               # Generates 2–4 sentence summary for RAG embedding
│
├── demographic_c/               # Component C — Demographic Agent
│   ├── agent.py                 # DemographicAgent class
│   ├── bio_parser.py            # Bio-text inference
│   ├── pattern_analyzer.py      # Posting-time-pattern analysis
│   ├── cluster_guard.py         # Minimum cluster size enforcement
│   └── summary.py
```

```
│
├── trend_d/                    # Component D — Trend Agent
│   ├── agent.py                # TrendAgent class
│   ├── topic_model.py          # BERTopic pipeline
│   ├── velocity.py             # Velocity and acceleration calculation
│   └── summary.py
│
├── network_e/                  # Component E — Network Agent
│   ├── agent.py                # NetworkAgent class
│   ├── graph_builder.py        # NetworkX graph construction
│   ├── centrality.py           # Centrality metric computation
│   ├── community.py            # Louvain community detection
│   ├── cib_signals.py          # CIB heuristic anomaly signal computation
│   └── summary.py
│
├── orchestrator/               # Orchestrator — coordinates all agents
│   ├── coordinator.py          # Main batch runner; fires agents in parallel
│   ├── batch.py                # Batch detection (pending rows → agent dispatch)
│   └── aggregator.py           # Cross-component insight aggregation
│
├── rag/                        # RAG Query Layer
│   ├── embedder.py             # Summary → embedding → pgvector storage
│   ├── retriever.py            # pgvector similarity search
│   ├── llm/
│   │   ├── base.py             # BaseLLMProvider abstract class
│   │   ├── ollama_provider.py  # Ollama backend
│   │   └── openai_provider.py  # OpenAI backend
│   ├── prompt.py               # Grounding system prompt (enforced, not optional)
│   └── citation_parser.py      # Parses [DOC_N] tags → source record references
│
├── api/                        # Backend API layer (FastAPI)
│   ├── main.py                 # FastAPI app initialization
│   ├── auth.py                 # HTTP Basic Auth middleware
│   ├── routers/
│   │   ├── overview.py         # /api/overview/ routes (cross-component KPIs)
│   │   ├── ingestion_a.py      # /api/ingestion-a/ routes
│   │   ├── sentiment_b.py      # /api/sentiment-b/ routes
│   │   ├── demographic_c.py    # /api/demographics-c/ routes
│   │   ├── trend_d.py          # /api/trends-d/ routes
│   │   ├── network_e.py        # /api/network-e/ routes
│   │   └── rag.py              # /api/rag/ routes
│   └── schemas/                # Pydantic request/response schemas (API contract)
│       ├── overview.py
│       ├── ingestion_a.py
│       ├── sentiment_b.py
│       ├── demographic_c.py
│       ├── trend_d.py
```

```
│       ├── network_e.py
│       └── rag.py
│
├── frontend/                    # Frontend (React + Vite + Tailwind)
│   ├── src/
│   │   ├── views/
│   │   │   ├── Overview.jsx
│   │   │   ├── SentimentTimeline.jsx    # Component B view
│   │   │   ├── TrendExplorer.jsx        # Component D view
│   │   │   ├── Demographics.jsx         # Component C view
│   │   │   ├── NetworkGraph.jsx         # Component E view
│   │   │   ├── DataSources.jsx          # Component A provenance view
│   │   │   └── QueryPanel.jsx           # RAG chat interface
│   │   ├── components/                  # Reusable UI components
│   │   └── api/                         # API client functions (built against schemas)
│   └── package.json
│
├── benchmarks/                  # Offline evaluation benchmarks (not in live pipeline)
│   ├── README.md
│   └── cib_ioa/
│       ├── ioa_users.csv        # Kaggle-sourced coordinated/organic ground-truth dataset (see Section 4.1a)
│       ├── aggregate_users.py   # Aggregates tweets to user-level profiles
│       └── evaluate.py          # Standalone precision/recall/F1 evaluation script
│
└── tests/
    ├── test_ingestion_a.py
    ├── test_sentiment_b.py
    ├── test_demographic_c.py
    ├── test_trend_d.py
    ├── test_network_e.py
    ├── test_orchestrator.py
    ├── test_rag.py
    └── test_api_endpoints.py
```

---

12. Docker Compose Services (Skeleton)

```
services:
  postgres:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  backend:
    build: .
    command: uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload
    environment:
      - DATABASE_URL=${DATABASE_URL}
      - REDIS_URL=${REDIS_URL}
      - LLM_PROVIDER=${LLM_PROVIDER}
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - OLLAMA_BASE_URL=${OLLAMA_BASE_URL}
      - API_USERNAME=${API_USERNAME}
      - API_PASSWORD=${API_PASSWORD}
    depends_on: [postgres, redis]
    ports:
      - "8000:8000"

  celery-worker:
    build: .
    # Concurrency=1 ensures Pyrofork SQLite session file is never accessed concurrently
    command: celery -A orchestrator.coordinator worker --concurrency=1 --loglevel=info
    environment:
      - DATABASE_URL=${DATABASE_URL}
      - REDIS_URL=${REDIS_URL}
    depends_on: [postgres, redis]

  celery-beat:
    build: .
    command: celery -A orchestrator.coordinator beat --loglevel=info
    environment:
```

```
      - DATABASE_URL=${DATABASE_URL}
      - REDIS_URL=${REDIS_URL}
      - INGESTION_A_INTERVAL_MINUTES=${INGESTION_A_INTERVAL_MINUTES:-15}
      - COMPONENT_B_INTERVAL_MINUTES=${COMPONENT_B_INTERVAL_MINUTES:-15}
      - COMPONENT_C_INTERVAL_MINUTES=${COMPONENT_C_INTERVAL_MINUTES:-60}
      - COMPONENT_D_INTERVAL_MINUTES=${COMPONENT_D_INTERVAL_MINUTES:-60}
      - COMPONENT_E_INTERVAL_MINUTES=${COMPONENT_E_INTERVAL_MINUTES:-120}
    depends_on: [postgres, redis]

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on: [backend]

  ollama:                          # Viva default LLM runtime
    image: ollama/ollama
    volumes:
      - ollama_data:/root/.ollama
    ports:
      - "11434:11434"

volumes:
  postgres_data:
  ollama_data:
```

---

13. Open Items — Confirm Before Phase 1 Starts

The execution sequencing below is strictly gated: **Gate 1 items represent non-negotiable institutional and compliance prerequisites** that must be formally settled before any operational scraping, channel locking, or phone number registration takes place.

Gate 1: Institutional Ethics & Compliance Sign-Off (Prerequisite Zero)

- [ ] **Institutional ethics/data-handling sign-off:** Confirm with project supervisor that research scope covers public Telegram broadcast channels. Document in project report.
- [ ] **Semester-end PII disposal commitment:** Codify simple post-viva scrub commitment (`UPDATE raw_posts SET content_text = NULL; UPDATE raw_authors SET display_name = NULL, bio_text = NULL;`) in project report.

Gate 2: Operational & Environmental Verification (Technical Setup)

- [ ] **Confirm topic domain** (e.g., ISRO/Indian space, Indian tech policy)
- [ ] **Team selects and documents 30 Telegram channels** (populate `config/channels.yaml`)
- [ ] **P1 installs and tests Pyrofork** with a dedicated project phone number (credentials in `.env`)
- [ ] **Assign component ownership** (P1: A/Infra, P2: B/D, P3: E/Orchestrator, P4: C/RAG)

- [ ] **Configure LLM in `.env`:** Install Ollama with `llama3.2:3b` as **viva default**. Groq/OpenAI keys are optional for development only.
- [ ] **Confirm evaluation benchmark access** (Hugging Face datasets for TweetEval / GoEmotions)
- [ ] **Confirm `ioa_users.csv` is correctly placed** in `benchmarks/cib_ioa/` in the repo, with its corrected provenance description (Section 4.1a) copied into `benchmarks/README.md`

Document version: 4.8 (Academic Prototype — Hardware Realism & 30-Channel Staggered Polling) | Last updated: October 2026 | Supersedes v4.7 — 30-channel staggered MTProto rate-limiting, DistilBART runtime default, Ollama viva LLM, GTX 1650 Ti VRAM sequential execution.